Operaciones de Aprendizaje Automático III

Jev y Destilación

MSc. Fernando Silva



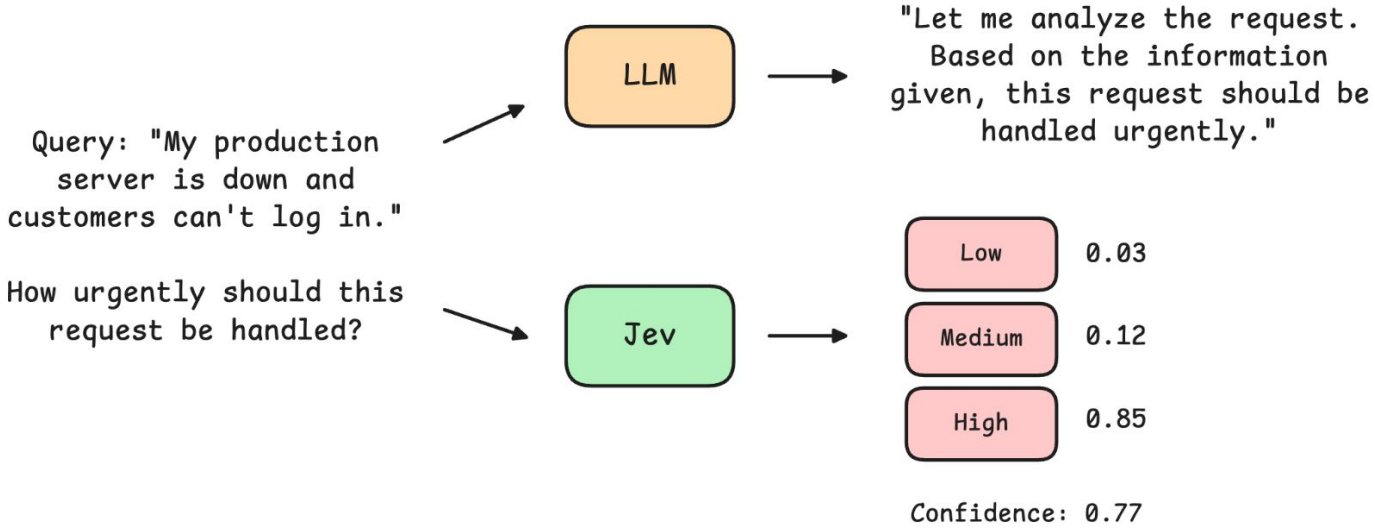
Agenda

- Jev
- Jev y Destilación

Jev

Jev

- Jev es un modelo de decisión de TypeSafe AI
- En vez de generar texto, devuelve una opción tipada con su probabilidad calibrada: elegir entre opciones, un puntaje, o sí/no
- Cuesta 0.042 dólares por millón de tokens de entrada



Jev

minicpm5-2b

- MiniCPM5-2B es el modelo base sobre el que se construyó el MiniCPM5-Jev
- Lo desarrollan OpenBMB, Modelbest, THUNLP y la Universidad Renmin (RUC), y se publicó el 7 de septiembre de 2026 bajo licencia Apache-2.0
- Está pensado para uso local, en dispositivos y con pocos recursos
- Arquitectura estándar LlamaForCausalLM, así que funciona en los motores habituales sin código especial.
- Unos 2.500 millones de parámetros, de los cuales cerca de 2.000 millones no son de embedding.
- 42 capas y atención GQA (16 cabezas de consulta, 2 de clave/valor).
- Contexto de 131.072 tokens.

Jev

minicpm5-2b-jev

- MiniCPM5-2B-Jev es un modelo de “decisión rápida” (System 1) de 2.000 millones de parámetros, basado en MiniCPM5-2B de OpenBMB y publicado bajo licencia Apache-2.0.

- Recibe un documento de contexto (el “estado”) y una o varias preguntas tipadas, y devuelve directamente una distribución de probabilidad calibrada sobre las opciones de cada pregunta, sin generar texto.

- Admite tres tipos de pregunta:
  - **choice**: elegir entre varias opciones, hasta 77 en sus pruebas.
  - **noul**: verdadero o falso.
  - **score**: puntuación ordinal, por ejemplo de 0 a 3.

- Es compatible con la API /v1/systemone que usan clientes como Jev, OpenJev, Decider y Kev.

Jev

minicpm5-2b-jev

- **Lectura por letras:**
  - proyecta el estado oculto final sobre los tokens de letra (A, B, C…) reutilizando pesos del propio modelo base
  - añade corrección de sesgos y temperaturas de calibración por tipo de pregunta.

- **Caché KV compartida:**
  - codifica el contexto una sola vez y evalúa cada pregunta en una rama aislada
  - así las preguntas no se influyen entre sí y la latencia es baja

- **Preprocesador de fechas:**
  - calcula automáticamente la diferencia en días entre las fechas del texto, porque a los modelos pequeños les cuesta hacerlo

- **Entrenamiento**:
  - usa LoRA (r=16, unos 25M de parámetros entrenables) sobre unos 18.600 ejemplos de cinco fuentes: datasets públicos de clasificación, razonamiento composicional, calibración y robustez, preguntas múltiples filtradas de un modelo maestro, y pares contrafactuales.



Ejercicio

Jev

LINK

Ollama

Ollama

Ollama es una herramienta que permite ejecutar modelos de lenguaje (LLM), directamente en tu computadora, sin necesidad de utilizar servicios externos en la nube.

Por ejemplo, permite descargar y ejecutar modelos como Llama, Qwen o Gemma en un equipo propio.

cómo funciona?

1. **Descarga**: elegimos y descargamos un modelo de lenguaje.
2. **Ejecución**: Ollama carga el modelo y utiliza los recursos de nuestra computadora, como la RAM y, cuando corresponde, la GPU.
3. **Interacción**: podemos hacer preguntas al modelo desde una terminal, una aplicación o mediante una API.

para qué sirve?

1. Ejecutar LLM localmente.
2. Desarrollar aplicaciones de IA sin depender de una API externa.
3. Crear chatbots y sistemas RAG.
4. Experimentar con modelos de lenguaje manteniendo los datos localmente.



Jev y Destilación

Contexto

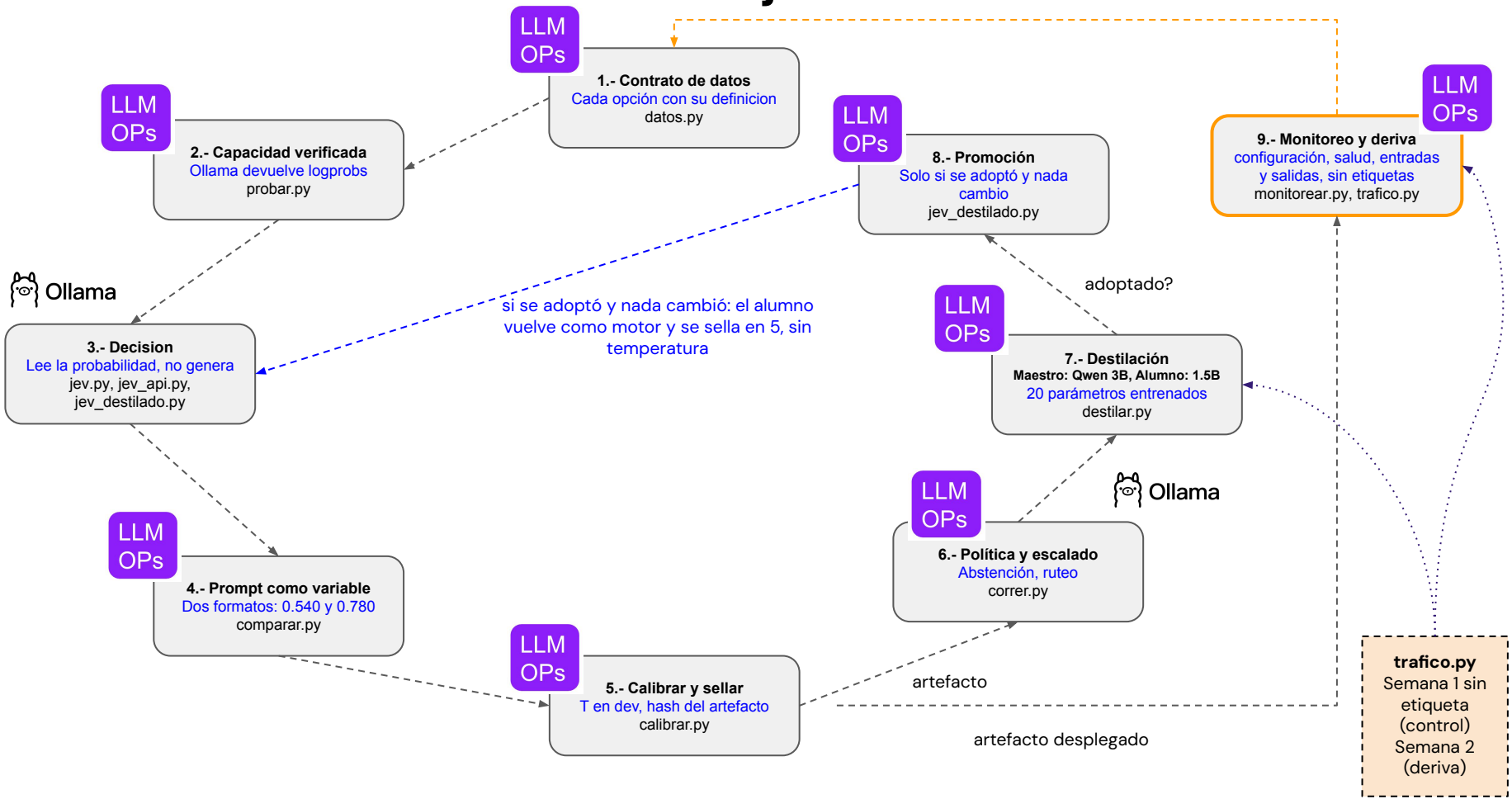
- **jev-simple** es un sistema que clasifica tickets de soporte en cuatro áreas: facturación, técnico, cuenta y otro, corre entero con un modelo pequeño de Ollama (qwen2.5 de 1.5B parámetros).
- **Idea central de Jev:** para decidir, el modelo no escribe una respuesta, se leen sus probabilidades.
- Un modelo que redacta "Esto parece de facturación" deja texto que hay que interpretar y que puede venir en cualquier formato. En cambio, si se le pide un solo carácter y se mira cuánta probabilidad le da a cada opción, sale una decisión con su grado de confianza, en una sola pasada y sin nada que interpretar.

Esa confianza permite que el sistema diga "no estoy seguro, que lo vea alguien más", y ahí empieza la parte de LLMOps:

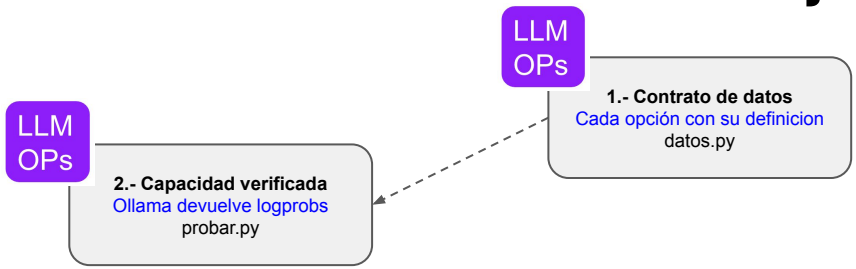
- medir si esa confianza es creíble
- decidir qué hacer con ella
- abaratar el sistema
- vigilarlo cuando ya está en uso

Flujo

si hay deriva: etiquetar una muestra, sumarla a datos.py y recalibrar



Flujo


LLM OPs
1.- Contrato de datos
Cada opción con su definicion
datos.py
LLM OPs
2.- Capacidad verificada
Ollama devuelve logprobs
probar.py

Flujo

Ticket A: “Quiero cancelar porque me parece caro.” El área correcta es cuenta.
Ticket B: “La factura no se descarga, tira error.” El área correcta es técnico.

datos.py

Contrato de datos:

```
AREAS = ["facturacion", "tecnico", "cuenta", "otro"]

SISTEMA = """Clasificas tickets de soporte en un area.

0 facturacion: cobros, importes, facturas, reembolsos, precios.
1 tecnico: la aplicacion falla, da error, no carga, va lenta.
2 cuenta: acceso, contrasena, datos personales, suscripcion, baja.
3 otro: cualquier otra cosa.

Un cargo mal hecho es facturacion, no cuenta.
No poder entrar es cuenta, no tecnico.
Responde solo con el numero del area entre corchetes."""
```

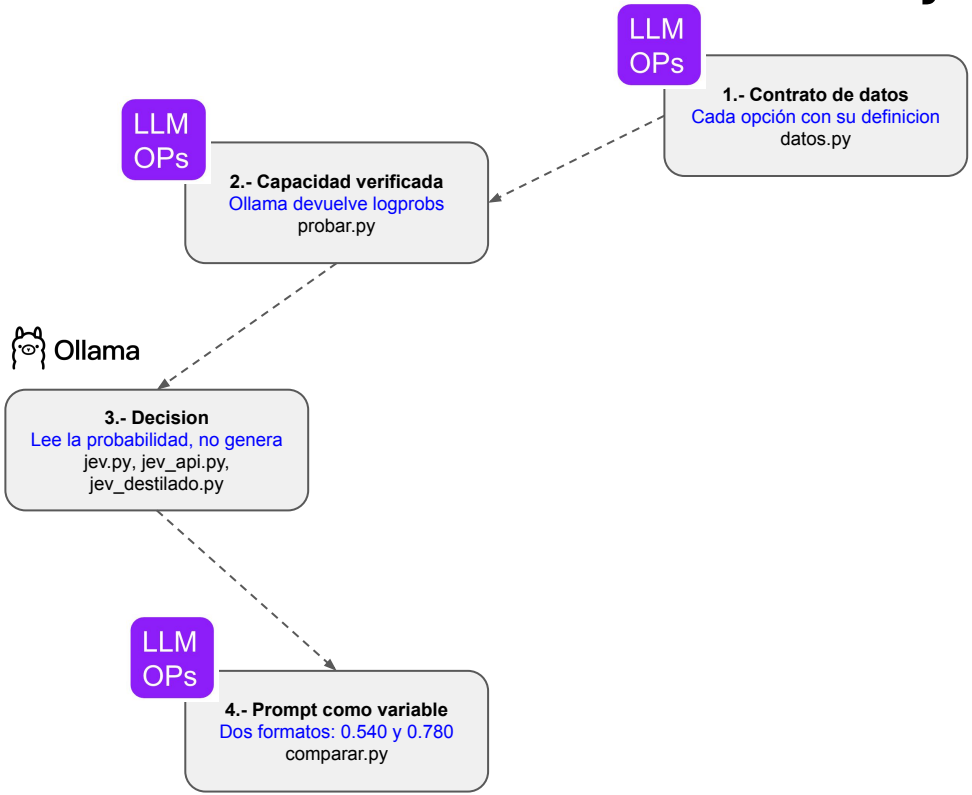
```
TICKETS = [
    ("Me cobraron 49 dolares y el plan cuesta 29.", 0),
    ("No carga el panel, queda en blanco.", 1),
    ("No puedo entrar, dice usuario o clave incorrectos.", 2),
    ("Quiero trabajar con ustedes, donde mando el cv?", 3),
...
50 tickets
...
]
```



probar.py

Se comprueba que Ollama responda, que tenga el modelo y que devuelva probabilidades por token

Flujo


LLM OPs
1.- Contrato de datos
Cada opción con su definicion
datos.py
LLM OPs
2.- Capacidad verificada
Ollama devuelve logprobs
probar.py
Ollama
3.- Decision
Lee la probabilidad, no genera
jev.py, jev_api.py,
jev_destilado.py
LLM OPs
4.- Prompt como variable
Dos formatos: 0.540 y 0.780
comparar.py

Flujo

Ticket A: “Quiero cancelar porque me parece caro.” El área correcta es cuenta.
Ticket B: “La factura no se descarga, tira error.” El área correcta es técnico.

jev.py

Ticket A entra en un prompt que termina en [.

<|im_start|>assistant\n[

Al modelo se le pide un solo token y se leen las probabilidades de los candidatos. Se descartan los que no son 0, 1, 2 o 3 y se renormaliza:

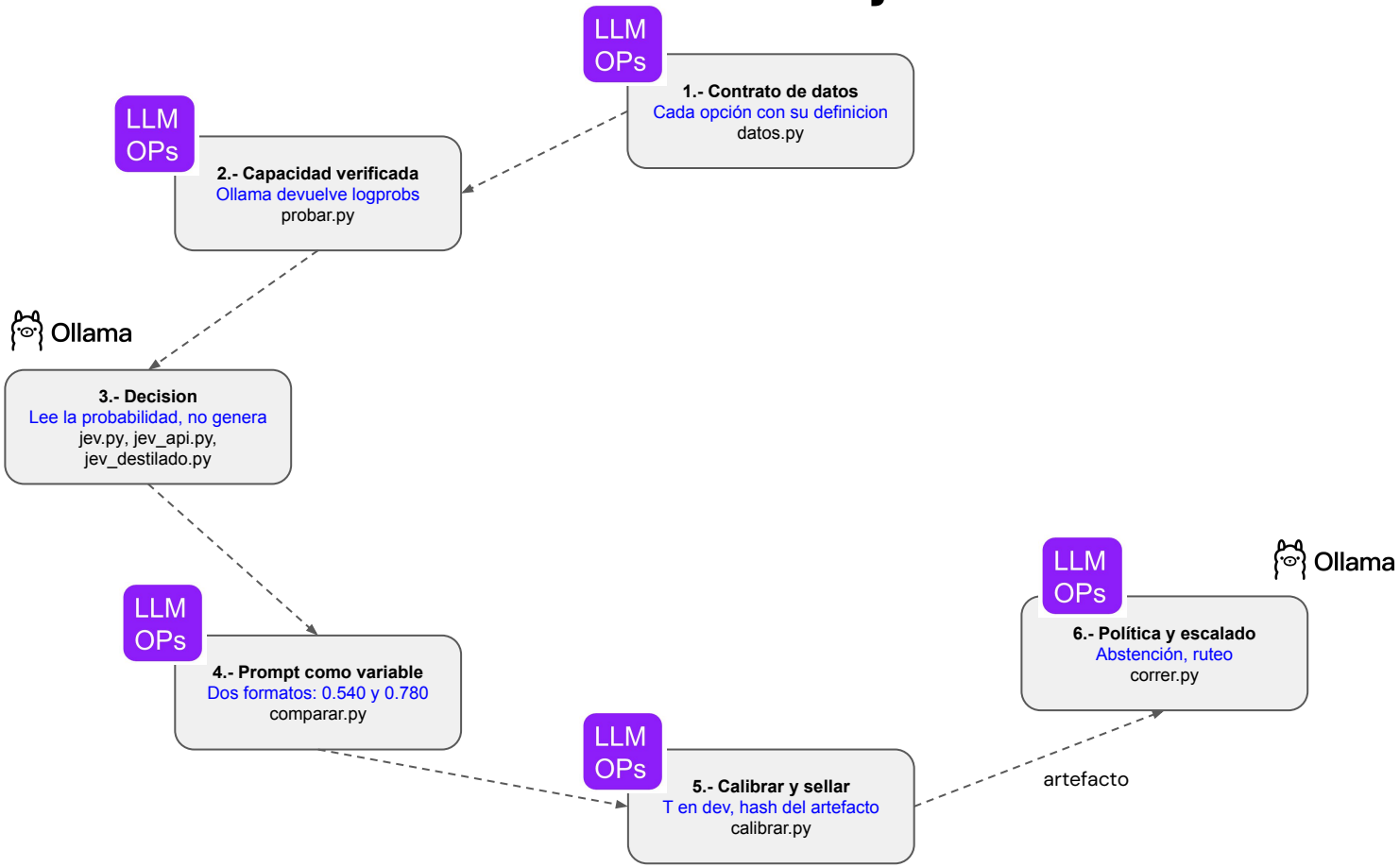
	facturación	técnico	cuenta	otro
ticket A	0.177	0.138	0.59	0.096
ticket B	0.784	0.192	0.022	0.002

El A acierta, pero duda.
El B se equivoca con bastante seguridad: leyó “factura” y no “error”.

comparar.py

- el mismo modelo, con el mismo texto, acierta 0.540 con el prompt en texto plano y 0.780 con el formato de chat
- por eso el prompt se trata como parte del modelo: se versiona y su huella entra al artefacto

Flujo


LLM OPs
1.- Contrato de datos
Cada opción con su definicion
datos.py
LLM OPs
2.- Capacidad verificada
Ollama devuelve logprobs
probar.py
Ollama
3.- Decision
Lee la probabilidad, no genera
jev.py, jev_api.py,
jev_destilado.py
LLM OPs
4.- Prompt como variable
Dos formatos: 0.540 y 0.780
comparar.py
LLM OPs
5.- Calibrar y sellar
T en dev, hash del artefacto
calibrar.py
artefacto
LLM OPs
Ollama
6.- Política y escalado
Abstención, ruteo
correr.py

Flujo

Ticket A: “Quiero cancelar porque me parece caro.” El área correcta es cuenta.
Ticket B: “La factura no se descarga, tira error.” El área correcta es técnico.

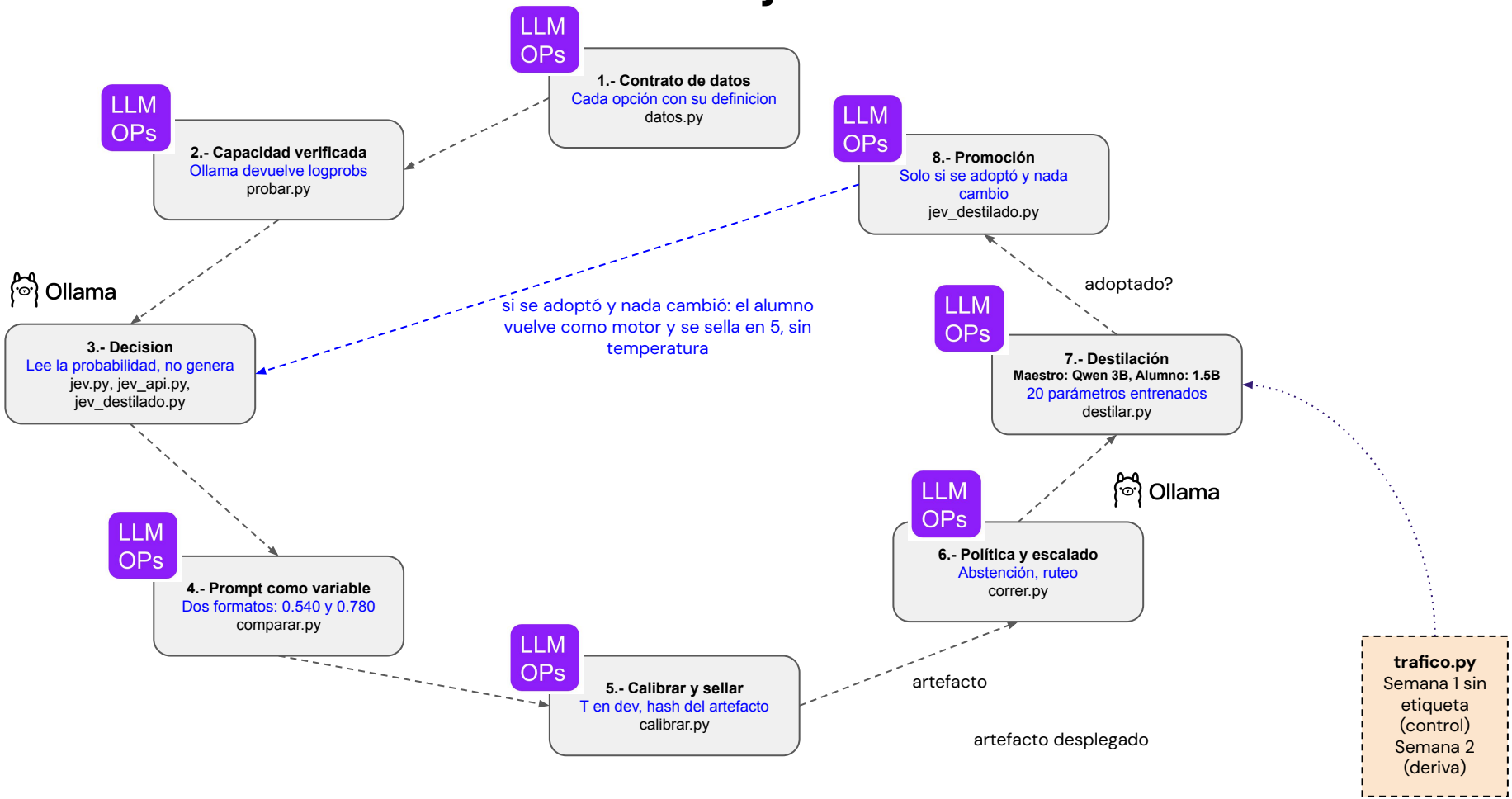
calibrar.py

- se deciden los 50 tickets y se busca, con los 30 de desarrollo, la temperatura que mejor corrige la confianza
- sale T = 1.1, que suaviza un poco: el A queda en 0.558 y el B en 0.757
- después todo se sella en artefacto.json (modelo, digest, prompt, temperatura y las 50 decisiones) con el hash f97255007d0c4748

correr.py

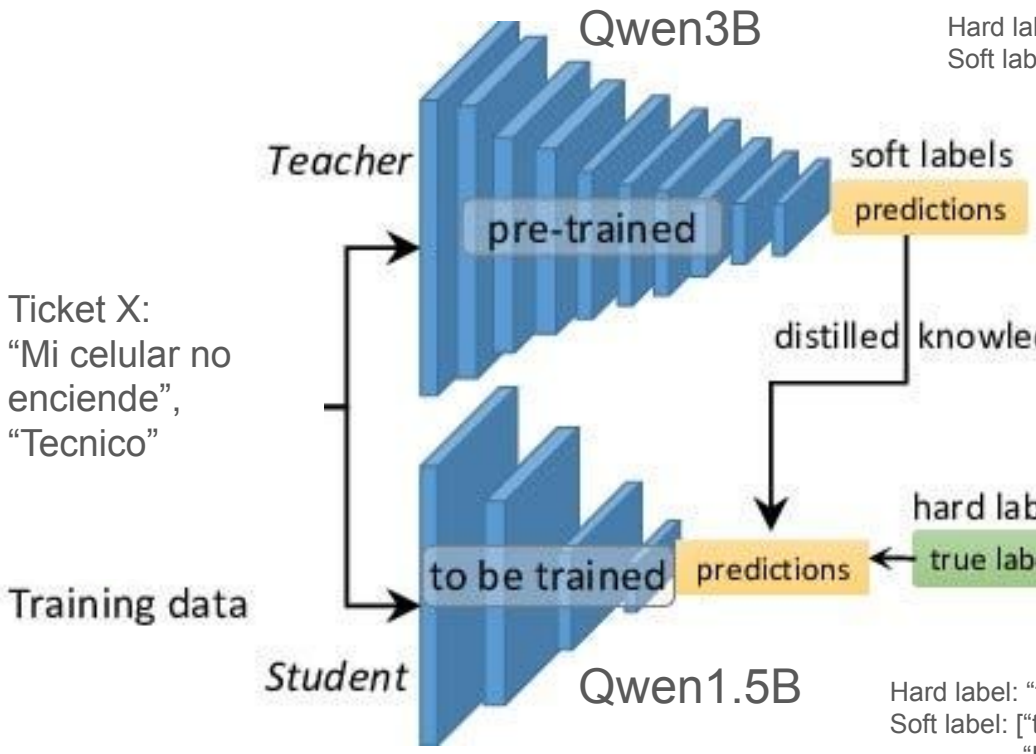
- si cambia cualquiera de esas cosas, cambia el hash
- **Ticket A: 0.558 < 0.75**, así que escala al LLM, que redactaría un borrador, como además está por debajo de 0.6, se marca para revisión humana
- **Ticket B: 0.757 ≥ 0.75**, así que se resuelve solo y mal, es el límite de cualquier umbral: no ve los errores que el modelo comete con seguridad

Flujo


LLM OPs
1.- Contrato de datos
Cada opción con su definicion
datos.py
LLM OPs
2.- Capacidad verificada
Ollama devuelve logprobs
probar.py
Ollama
3.- Decision
Lee la probabilidad, no genera
jev.py, jev_api.py,
jev_destilado.py
LLM OPs
4.- Prompt como variable
Dos formatos: 0.540 y 0.780
comparar.py
LLM OPs
5.- Calibrar y sellar
T en dev, hash del artefacto
calibrar.py
artefacto
LLM OPs
6.- Política y escalado
Abstención, ruteo
correr.py
Ollama
LLM OPs
7.- Destilación
Maestro: Qwen 3B, Alumno: 1.5B
20 parámetros entrenados
destilar.py
adoptado?
LLM OPs
8.- Promoción
Solo si se adoptó y nada cambio
jev_destilado.py
si se adoptó y nada cambió: el alumno vuelve como motor y se sella en 5, sin temperatura
artefacto desplegado
trafico.py
Semana 1 sin etiqueta (control)
Semana 2 (deriva)

Destilación (Knowledge Distillation)

Desplegado:
Qwen1.5B



Hard label: “tecnico”
Soft label: [“tecnico”: 0.70,
“Cuenta”: 0.11,
“Facturacion: 0.10,
“Otro”: 0.09]

C.E.: hard labels

KL: soft labels

Hard label: “tecnico”
Soft label: [“tecnico”: 0.55,
“Facturacion”: 0.31,
“Otro: 0.10,
“Cuenta”: 0.04]

Flujo

Ticket A: “Quiero cancelar porque me parece caro.” El área correcta es cuenta.
Ticket B: “La factura no se descarga, tira error.” El área correcta es técnico.

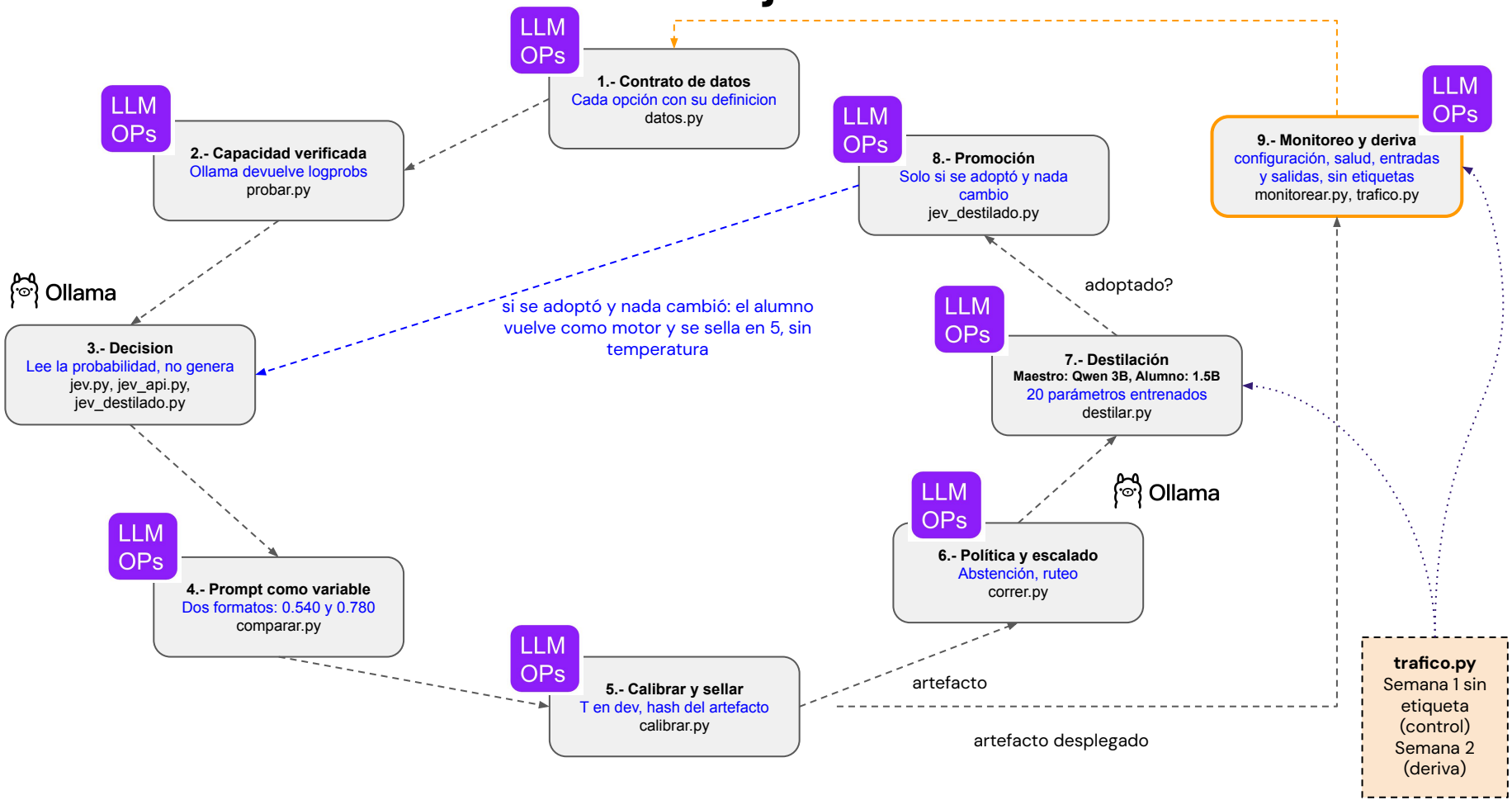
destilar.py con la semana 1 de trafico.py

	modelo	prompt	accuracy en prueba
maestro	qwen 3B	con 4 ejemplos	0.8
desplegado	qwen 1.5B	con 4 ejemplos	0.7
alumno	qwen 1.5B	sin ejemplos, 139 tokens	0.550, y 0.700 con el mapa

- se entrenan 20 parámetros para que las probabilidades del alumno se parezcan a las del maestro
- se usan los tickets de desarrollo (el A está ahi) y los 30 de la semana 1, sin etiqueta
- el B está en prueba, así que solo sirve para evaluar
- Se revisan criterios (no pierde más e un ticket de accuracy, su confianza es más creible, ahorra tokens y el costo total no sube) y se decide que **no se adopta.** El alumno acierta lo mismo y es más barato por decisión pero duda más, y cada duda se paga con el LLM

Flujo

si hay deriva: etiquetar una muestra, sumarla a datos.py y recalibrar



Flujo

Ticket A: “Quiero cancelar porque me parece caro.” El área correcta es cuenta.
Ticket B: “La factura no se descarga, tira error.” El área correcta es técnico.

jev_destilado.py

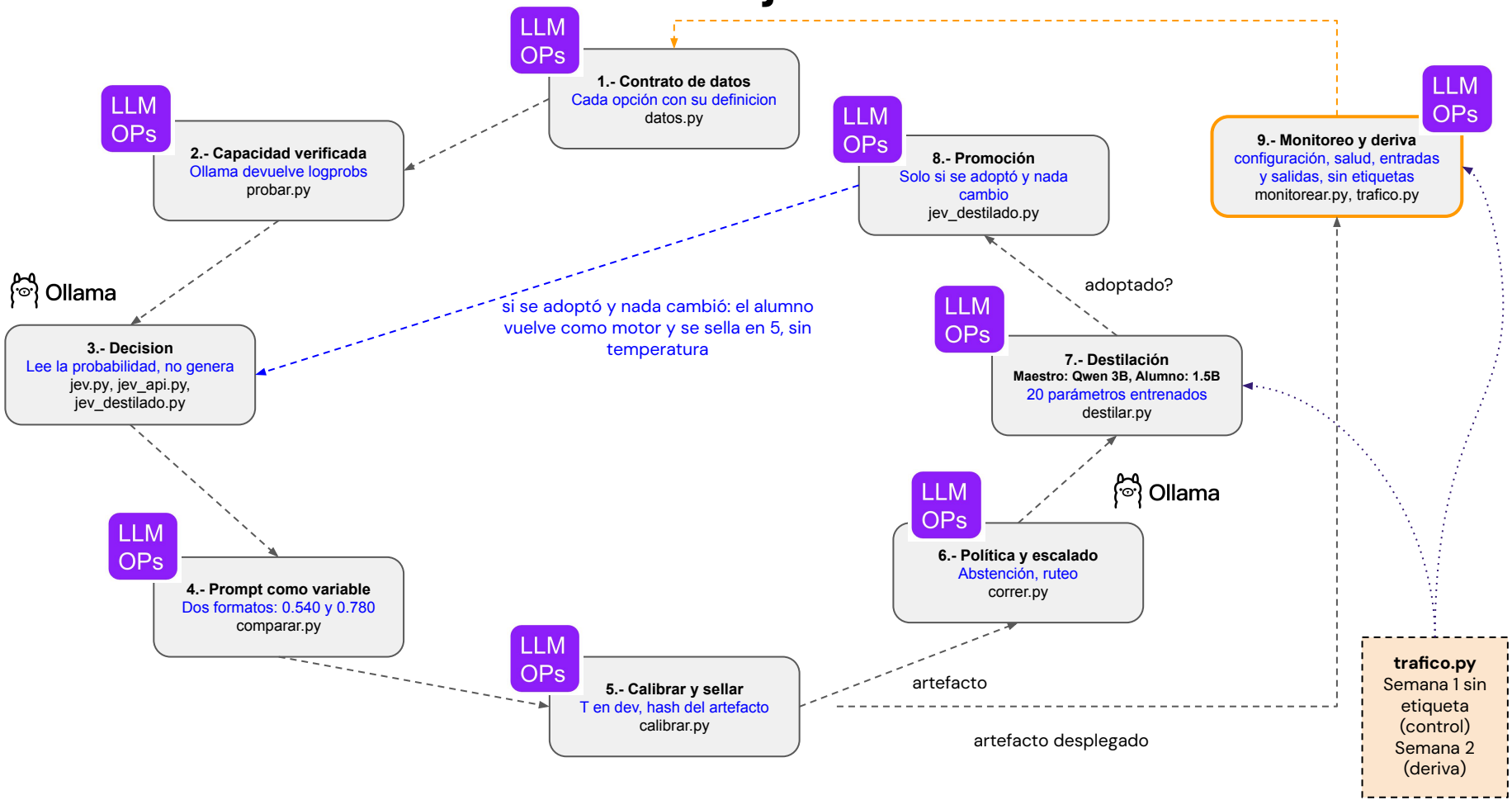
- si se hubiera adoptado, el alumno reemplazaría a lo desplegado como motor en el bloque 3 y se volvería a sellar en el 5, sin temperatura
- como no se adoptó (falló el costo total), el control lo frena
- hace cumplir el criterio de destilar.py: solo deja que el alumno reemplace a lo desplegado si fue adoptado, si el modelo es el mismo y si el prompt corto no cambió

monitorear.py con **trafico.py**

- Se vigila la configuración, las entradas y las salidas, comparadas con los 50 tickets de referencia del artefacto desplegado.
- Semana 1 (control): las cuatro señales salen estables
- Semana 2: las entradas dan deriva ($p < 0.001$). Por ejemplo “El QR acepta billeteras virtuales?” tuvo 0.379 y escaló porque es un producto que el modelo no conoce
- Si llegan tickets distintos a los de siempre el monitor avisa.

Flujo

si hay deriva: etiquetar una muestra, sumarla a datos.py y recalibrar





Ejercicio

Jev y Destilación

LINK